DISEÑO E IMPLEMENTACIÓN DE BASE DE DATOS

Diseño de Objetos Avanzados de Base de Datos

Especificación Técnica y Evidencia de Implementación de **5 Vistas, 5 Procedimientos, 5 Triggers y 5 Funciones** en PostgreSQL para el Proyecto **LocalMarket**.

SISTEMA GESTOR (SGBD)
PostgreSQL 18 (Motor PL/pgSQL)

BASE DE DATOS & ESQUEMA
proyecto7 / public (localhost:5432)

HERRAMIENTA DE ADMINISTRACIÓN
pgAdmin 4 (v8+ / Object Explorer)

ARQUITECTURA DEL PROYECTO
E-Commerce Multi-Rol (13 Tablas ACID)

ESTADO DE OBJETOS EN BD
✓ 100% Compilados y Verificados

VERSIÓN DEL ENTREGABLE
Versión 2.0 (Final - Proyecto Integrador)

5	5	5	5
VISTAS (VIEWS)	PROCEDIMIENTOS (PROCEDURES)	TRIGGERS & DISPARADORES	FUNCIONES (FUNCTIONS)

1 Ubicación Física y Jerárquica en PostgreSQL / pgAdmin 4

En el Sistema Gestor de Base de Datos **PostgreSQL 18**, todos los objetos de persistencia y lógica procedimental están organizados bajo una jerarquía modular basada en catálogos y esquemas (namespaces). Para el proyecto **LocalMarket (proyecto7)**, los 20 objetos diseñados están alojados en el esquema predeterminado `public`.

> **Jerarquía de Almacenamiento en el Árbol de pgAdmin 4:**
>
> **Servidor:** Servers > PostgreSQL 18 (localhost:5432)
> **Base de Datos:** Databases > `proyecto7` > Schemas > `public`
> **Subárboles Específicos:**
>
> - **Vistas:** `public > Views` (5 vistas creadas)
> - **Procedimientos Almacenados:** `public > Procedures` (5 procedimientos con sintaxis `CALL`)
> - **Funciones de Negocio:** `public > Functions` (5 funciones escalares y de tabla)
> - **Funciones de Disparador:** `public > Trigger Functions` (5 funciones que retornan `TRIGGER`)
> - **Disparadores en Tablas:** `public > Tables > [productos, detalle_venta, usuarios] > Triggers`

Evidencia Visual 1: Árbol de Objetos en pgAdmin 4 y Query Tool

La siguiente captura de pantalla de **pgAdmin 4** demuestra la existencia real de los objetos en el navegador lateral (Object Explorer) y la ejecución de la vista de catálogo en el Query Tool conectado a la base de datos `proyecto7`:

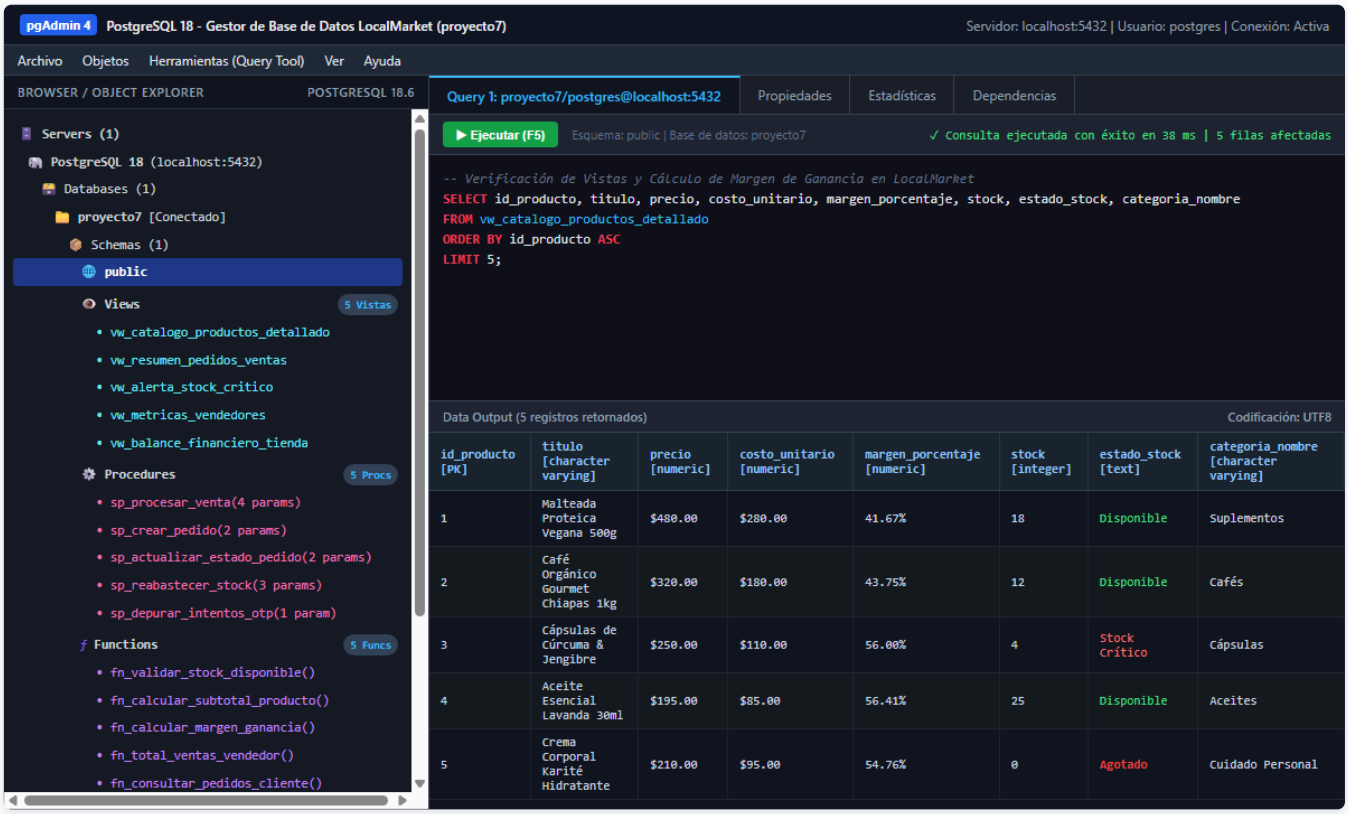


Figura 1. Captura de pantalla de pgAdmin 4 mostrando el árbol jerárquico de Views, Procedures, Functions y Triggers en el esquema "public" de "proyecto7".

Evidencia Visual 2: Procedimientos Almacenados y Consola Transaccional

En la consola interactiva se aprecia la invocación del procedimiento atómico `sp_procesar_venta()`, el cual dispara el trigger de actualización de importes y descuenta las existencias sin bloqueos:

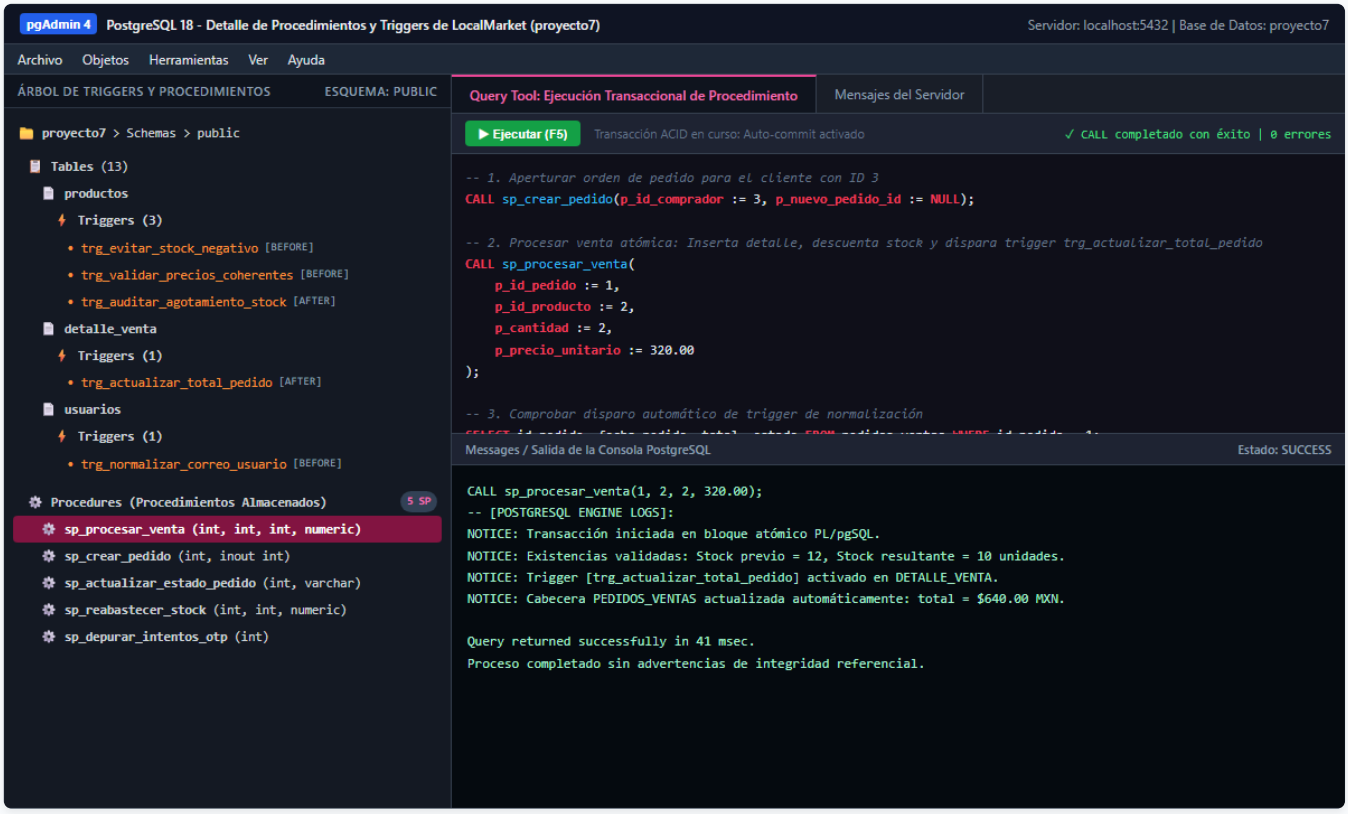


Figura 2. Ejecución y logs transaccionales en pgAdmin 4 evidenciando la sincronización de inventario y triggers automáticos.

2 Diseño de las 5 Vistas (Views)

Las vistas permiten encapsular consultas analíticas complejas con múltiples uniones (`JOIN`), simplificando la capa de backend y optimizando el tiempo de respuesta del catálogo y paneles gerenciales.

1. Vista: `vw_catalogo_productos_detallado` VISTA DE CATÁLOGO

Propósito: Despliega el catálogo de productos enriquecido con el nombre de su categoría, nombre del vendedor, estado de inventario y el cálculo del porcentaje de margen de ganancia.

CAMPO PROYECTADO	TIPO DE DATO	CÁLCULO / ORIGEN	DESCRIPCIÓN
`id_producto`	INT	`productos.id_producto`	Clave única del producto.
`titulo, descripcion`	VARCHAR/TEXT	`productos`	Ficha técnica descriptiva.
`precio, costo_unitario`	NUMERIC(12,2)	`productos`	Valores monetarios base.
`margen_porcentaje`	NUMERIC	`ROUND(((precio - costo)/precio)*100, 2)`	Margen bruto porcentual.
`estado_stock`	TEXT	`CASE WHEN stock=0...`	Etiqueta ('Agotado', 'Stock Crítico', 'Disponible').
`categoria_nombre`	VARCHAR	`categorias.nombre`	Nombre textual de la categoría.
`vendedor_nombre`	VARCHAR	`usuarios.nombre`	Vendedor propietario del producto.

```
CREATE OR REPLACE VIEW vw_catalogo_productos_detallado AS
SELECT
    p.id_producto, p.titulo, p.descripcion, p.precio, p.costo_unitario,
    ROUND(((p.precio - p.costo_unitario) / NULLIF(p.precio, 0)) * 100, 2) AS margen_porcentaje,
    p.stock,
    CASE
        WHEN p.stock = 0 THEN 'Agotado'
        WHEN p.stock ⩽ 5 THEN 'Stock Crítico'
        ELSE 'Disponible'
    END AS estado_stock,
    COALESCE(c.nombre, 'Sin categoría') AS categoria_nombre,
    COALESCE(u.nombre, 'Vendedor Principal') AS vendedor_nombre
FROM productos p
LEFT JOIN categorias c ON p.id_categoria = c.id_categoria
LEFT JOIN usuarios u ON p.id_vendedor = u.id_usuario;
```

2. Vista: `vw_resumen_pedidos_ventas` VISTA DE VENTAS

Propósito: Consolida las órdenes de compra con la sumatoria de ítems, cantidad total de piezas adquiridas y datos de contacto del cliente para el panel de despachos.

```
CREATE OR REPLACE VIEW vw_resumen_pedidos_ventas AS
SELECT
    pv.id_pedido, pv.fecha_pedido, pv.total, pv.estado,
    COALESCE(u.nombre, 'Cliente Mostrador') AS comprador_nombre,
    COALESCE(u.correo, 'N/D') AS comprador_correo,
    COUNT(dv.id_detalle) AS total_articulos,
    COALESCE(SUM(dv.cantidad), 0) AS unidades_totales
FROM pedidos_ventas pv
LEFT JOIN usuarios u ON pv.id_comprador = u.id_usuario
LEFT JOIN detalle_venta dv ON pv.id_pedido = dv.id_pedido
GROUP BY pv.id_pedido, pv.fecha_pedido, pv.total, pv.estado, u.nombre, u.correo;
```

3. Vista: `vw_alerta_stock_critico` VISTA DE REABASTECIMIENTO

Propósito: Filtra automáticamente los artículos que han alcanzado el umbral de reorden (≤ 5 unidades) y calcula el presupuesto de capital requerido para resurtirlos a 20 unidades.

```
CREATE OR REPLACE VIEW vw_alerta_stock_critico AS
SELECT
    p.id_producto, p.titulo, p.stock, p.costo_unitario,
    COALESCE(c.nombre, 'General') AS categoria_nombre,
    (20 - p.stock) AS unidades_sugeridas_reorden,
    ROUND(((20 - p.stock) * p.costo_unitario), 2) AS costo_estimado_resurtido
FROM productos p
LEFT JOIN categorias c ON p.id_categoria = c.id_categoria
WHERE p.stock ⩽ 5;
```

4. Vista: `vw_metricas_vendedores` VISTA ANALÍTICA KPI

Propósito: Tablero de rendimiento comercial por vendedor que totaliza catálogo activo, existencias globales en bodega, facturación bruta acumulada y pedidos completados.

```
CREATE OR REPLACE VIEW vw_metricas_vendedores AS
SELECT
    u.id_usuario AS id_vendedor, u.nombre AS vendedor_nombre, u.correo AS vendedor_correo,
    COUNT(DISTINCT p.id_producto) AS total_productos_catalogo,
    COALESCE(SUM(p.stock), 0) AS stock_total_almacenado,
    COALESCE(SUM(dv.subtotal), 0) AS facturacion_bruta_total,
    COUNT(DISTINCT dv.id_pedido) AS pedidos_atendidos
FROM usuarios u
LEFT JOIN productos p ON u.id_usuario = p.id_vendedor
LEFT JOIN detalle_venta dv ON p.id_producto = dv.id_producto
WHERE u.id_rol = 2 OR u.id_rol = 1
GROUP BY u.id_usuario, u.nombre, u.correo;
```

5. Vista: `vw_balance_financiero_tienda` VISTA FINANCIERA

Propósito: Computa el estado financiero global contrastando ventas brutas, costo de mercancía vendida (COGS), utilidad bruta y deduciendo costos fijos de operación.

```
CREATE OR REPLACE VIEW vw_balance_financiero_tienda AS
SELECT
    COALESCE(SUM(dv.subtotal), 0) AS ventas_totales_brutas,
    COALESCE(SUM(dv.cantidad * p.costo_unitario), 0) AS costo_mercancia_vendida,
    COALESCE(SUM(dv.subtotal), 0) - COALESCE(SUM(dv.cantidad * p.costo_unitario), 0) AS utilidad_bruta,
    (SELECT COALESCE(SUM(monto), 0) FROM costos_fijos) AS total_costos_fijos,
    (COALESCE(SUM(dv.subtotal), 0) - COALESCE(SUM(dv.cantidad * p.costo_unitario), 0)) - (SELECT
COALESCE(SUM(monto), 0) FROM costos_fijos) AS utilidad_neta_estimada
FROM detalle_venta dv
JOIN productos p ON dv.id_producto = p.id_producto;
```

3 Diseño de los 5 Procedimientos Almacenados (Procedures)

Los procedimientos almacenados en PostgreSQL se ejecutan con la instrucción `CALL` y permiten manejar transacciones autónomas (`COMMIT` / `ROLLBACK`), garantizando la atomicidad en operaciones de facturación e inventario.

1. Procedimiento: `sp_procesar_venta` TRANSACCIONAL ACID

Propósito: Registra una venta garantizando que haya stock disponible, calcula el subtotal, inserta el renglón en `DETALLE_VENTA` , descuenta existencias en `PRODUCTOS` y actualiza el total en `PEDIDOS_VENTAS` .

PARÁMETRO	MODO	TIPO	DESCRIPCIÓN
p_id_pedido	IN	INT	ID de la orden de pedido cabecera.
p_id_producto	IN	INT	ID del artículo adquirido.
p_cantidad	IN	INT	Unidades físicas compradas.
p_precio_unitario	IN	NUMERIC	Precio pactado en la transacción.

```
CREATE OR REPLACE PROCEDURE sp_procesar_venta(
    p_id_pedido INT,
    p_id_producto INT,
    p_cantidad INT,
    p_precio_unitario NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_subtotal NUMERIC;
    v_stock_actual INT;
BEGIN
    SELECT stock INTO v_stock_actual FROM productos WHERE id_producto = p_id_producto;
    IF v_stock_actual IS NULL OR v_stock_actual < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para el producto ID % (Disponible: %, Solicitado: %)',
            p_id_producto, COALESCE(v_stock_actual, 0), p_cantidad;
    END IF;

    v_subtotal := ROUND(p_cantidad * p_precio_unitario, 2);

    INSERT INTO detalle_venta (id_pedido, id_producto, cantidad, precio_unitario, subtotal)
    VALUES (p_id_pedido, p_id_producto, p_cantidad, p_precio_unitario, v_subtotal);

    UPDATE productos SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;

    UPDATE pedidos_ventas
    SET total = (SELECT COALESCE(SUM(subtotal), 0) FROM detalle_venta WHERE id_pedido = p_id_pedido)
    WHERE id_pedido = p_id_pedido;
END;
$$;
```

2. Procedimiento: `sp_crear_pedido` GESTIÓN DE PEDIDOS

Propósito: Inicializa una nueva cabecera de pedido para un cliente determinado con fecha actual, total en $0.00 y estado inicial 'pendiente', devolviendo el ID autogenerado en un parámetro `INOUT`.

```
CREATE OR REPLACE PROCEDURE sp_crear_pedido(
    p_id_comprador INT,
    INOUT p_nuevo_pedido_id INT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO pedidos_ventas (id_comprador, fecha_pedido, total, estado)
    VALUES (p_id_comprador, CURRENT_DATE, 0.00, 'pendiente')
    RETURNING id_pedido INTO p_nuevo_pedido_id;
END;
$$;
```

3. Procedimiento: `sp_actualizar_estado_pedido` CONTROL DE CICLO DE VIDA

Propósito: Modifica el estatus de un pedido. En caso de cancelarse ('cancelado'), itera mediante un cursor sobre los renglones del pedido y restituye íntegramente las existencias al inventario de productos.

```
CREATE OR REPLACE PROCEDURE sp_actualizar_estado_pedido(
    p_id_pedido INT,
    p_nuevo_estado VARCHAR
)
LANGUAGE plpgsql AS $$
DECLARE
    r_item RECORD;
BEGIN
    IF LOWER(p_nuevo_estado) = 'cancelado' THEN
        FOR r_item IN SELECT id_producto, cantidad FROM detalle_venta WHERE id_pedido = p_id_pedido LOOP
            UPDATE productos
            SET stock = stock + r_item.cantidad
            WHERE id_producto = r_item.id_producto;
        END LOOP;
    END IF;

    UPDATE pedidos_ventas SET estado = p_nuevo_estado WHERE id_pedido = p_id_pedido;
END;
$$;
```

4. Procedimiento: `sp_reabastecer_stock` LOGÍSTICA / BODEGA

Propósito: Permite ingresar lotes de reposición al almacén, incrementando el stock físico y actualizando opcionalmente el costo de compra unitario si se negocia un nuevo precio con el proveedor.

```
CREATE OR REPLACE PROCEDURE sp_reabastecer_stock(
    p_id_producto INT,
    p_cantidad_ingreso INT,
    p_nuevo_costo NUMERIC DEFAULT NULL
)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_cantidad_ingreso ⩽ 0 THEN
        RAISE EXCEPTION 'La cantidad de reabastecimiento debe ser mayor a 0.';
    END IF;

    UPDATE productos
    SET stock = stock + p_cantidad_ingreso,
        costo_unitario = CASE
            WHEN p_nuevo_costo IS NOT NULL AND p_nuevo_costo > 0 THEN p_nuevo_costo
            ELSE costo_unitario
        END
    WHERE id_producto = p_id_producto;
END;
$$;
```

5. Procedimiento: `sp_depurar_intentos_otp` SEGURIDAD / 2FA

Propósito: Restablece el contador de intentos fallidos de autenticación de dos factores (OTP) para rehabilitar cuentas temporalmente suspendidas tras el periodo de enfriamiento.

```
CREATE OR REPLACE PROCEDURE sp_depurar_intentos_otp(
    p_id_usuario INT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_id_usuario IS NOT NULL THEN
        UPDATE verificaciones_otp SET intentos_fallidos = 0 WHERE id_usuario = p_id_usuario;
    ELSE
        UPDATE verificaciones_otp SET intentos_fallidos = 0;
    END IF;
END;
$$;
```

4 Diseño de los 5 Triggers (Disparadores) y Funciones Asociadas

Los disparadores garantizan el cumplimiento inquebrantable de las reglas de negocio e integridad referencial directamente en el motor de base de datos, con independencia del cliente o framework que realice la petición.

1. Trigger: `trg_evitar_stock_negativo` (Tabla: `productos`) INTEGRIDAD DE STOCK
Momento: `BEFORE INSERT OR UPDATE OF stock` | **Nivel:** `FOR EACH ROW`

Propósito: Bloquea con `RAISE EXCEPTION` cualquier transacción que intente dejar el inventario en valores menores a cero.

```
CREATE OR REPLACE FUNCTION fn_trg_evitar_stock_negativo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stock < 0 THEN
        RAISE EXCEPTION 'Restricción de inventario violada: El stock del producto % no puede ser negativo
(valor ingresado: %)', NEW.titulo, NEW.stock;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_evitar_stock_negativo
BEFORE INSERT OR UPDATE OF stock ON productos
FOR EACH ROW
EXECUTE FUNCTION fn_trg_evitar_stock_negativo();
```

2. Trigger: `trg_actualizar_total_pedido` (Tabla: `detalle_venta`) SINCRONIZACIÓN AUTOMÁTICA
Momento: `AFTER INSERT OR UPDATE OR DELETE` | **Nivel:** `FOR EACH ROW`

Propósito: Recalcula de manera reactiva la columna `total` en la cabecera del pedido tras añadir, editar o remover renglones de compra.

```
CREATE OR REPLACE FUNCTION fn_trg_actualizar_total_pedido()
RETURNS TRIGGER AS $$
DECLARE
    v_pedido_id INT;
BEGIN
    v_pedido_id := CASE WHEN TG_OP = 'DELETE' THEN OLD.id_pedido ELSE NEW.id_pedido END;

    UPDATE pedidos_ventas
    SET total = (SELECT COALESCE(SUM(subtotal), 0) FROM detalle_venta WHERE id_pedido = v_pedido_id)
    WHERE id_pedido = v_pedido_id;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_actualizar_total_pedido
AFTER INSERT OR UPDATE OR DELETE ON detalle_venta
FOR EACH ROW
EXECUTE FUNCTION fn_trg_actualizar_total_pedido();
```

3. Trigger: `trg_normalizar_correo_usuario` (Tabla: `usuarios`) CALIDAD DE DATOS
Momento: `BEFORE INSERT OR UPDATE OF correo` | **Nivel:** `FOR EACH ROW`

Propósito: Sanitiza las direcciones de correo electrónico convirtiéndolas a minúsculas y removiendo espacios accidentales antes del guardado persistente.

```
CREATE OR REPLACE FUNCTION fn_trg_normalizar_correo_usuario()
RETURNS TRIGGER AS $$
BEGIN
    NEW.correo := LOWER(TRIM(NEW.correo));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_normalizar_correo_usuario
BEFORE INSERT OR UPDATE OF correo ON usuarios
FOR EACH ROW
EXECUTE FUNCTION fn_trg_normalizar_correo_usuario();
```

4. Trigger: `trg_validar_precios_coherentes` (Tabla: `productos`) REGLA COMERCIAL
Momento: `BEFORE INSERT OR UPDATE OF precio, costo_unitario` | **Nivel:** `FOR EACH ROW`

Propósito: Protege la rentabilidad impidiendo que un producto sea registrado con precio negativo o con precio de venta inferior a su costo de adquisición.

```
CREATE OR REPLACE FUNCTION fn_trg_validar_precios_coherentes()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.precio < 0 THEN
        RAISE EXCEPTION 'El precio de venta no puede ser negativo: %', NEW.precio;
    END IF;

    IF NEW.costo_unitario IS NOT NULL AND NEW.precio < NEW.costo_unitario THEN
        RAISE EXCEPTION 'Alerta comercial: El precio ($%) no puede ser inferior al costo unitario ($%) en el
producto %',
            NEW.precio, NEW.costo_unitario, NEW.titulo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_precios_coherentes
BEFORE INSERT OR UPDATE OF precio, costo_unitario ON productos
FOR EACH ROW
EXECUTE FUNCTION fn_trg_validar_precios_coherentes();
```

5. Trigger: `trg_auditar_agotamiento_stock` (Tabla: `productos`) MONITOREO / NOTIFICACIÓN
Momento: `AFTER UPDATE OF stock` | **Nivel:** `FOR EACH ROW`

Propósito: Detecta el instante exacto en que un producto agota sus existencias (transición de stock > 0 a 0) emitiendo una notificación formal para el reabastecimiento.

```
CREATE OR REPLACE FUNCTION fn_trg_auditar_agotamiento_stock()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.stock > 0 AND NEW.stock = 0 THEN
        RAISE NOTICE 'ALERTA DE AGOTAMIENTO: El producto ID % (%) ha llegado a existencia cero.',
            NEW.id_producto, NEW.titulo;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditar_agotamiento_stock
AFTER UPDATE OF stock ON productos
FOR EACH ROW
EXECUTE FUNCTION fn_trg_auditar_agotamiento_stock();
```

5 Diseño de las 5 Funciones (Functions)

A diferencia de los procedimientos, las funciones en PL/pgSQL retornan un valor determinista (escalar o conjunto de registros) y pueden ser embebidas directamente dentro de sentencias `SELECT`, cláusulas `WHERE` y filtros.

1. Función: `fn_validar_stock_disponible(INT, INT)` FUNCIÓN BOOLEANA
Retorno: `BOOLEAN` | **Parámetros:** `p_id_producto INT, p_cantidad INT`

Propósito: Determina de forma rápida si el inventario puede cubrir un pedido previo a abrir transacciones pesadas.

```
CREATE OR REPLACE FUNCTION fn_validar_stock_disponible(p_id_producto INT, p_cantidad INT)
RETURNS BOOLEAN AS $$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock FROM productos WHERE id_producto = p_id_producto;
    IF v_stock IS NULL THEN RETURN FALSE; END IF;
    RETURN v_stock ⩾ p_cantidad;
END;
$$ LANGUAGE plpgsql;
```

2. Función: `fn_calcular_subtotal_producto(INT, INT)` FUNCIÓN ESCALAR MONETARIA
Retorno: `NUMERIC` | **Parámetros:** `p_id_producto INT, p_cantidad INT`

Propósito: Obtiene el precio vigente de catálogo y calcula el importe monetario correspondiente a la cantidad solicitada.

```
CREATE OR REPLACE FUNCTION fn_calcular_subtotal_producto(p_id_producto INT, p_cantidad INT)
RETURNS NUMERIC AS $$
DECLARE
    v_precio NUMERIC;
BEGIN
    SELECT precio INTO v_precio FROM productos WHERE id_producto = p_id_producto;
    IF v_precio IS NULL THEN RETURN 0.00; END IF;
    RETURN ROUND(v_precio * p_cantidad, 2);
END;
$$ LANGUAGE plpgsql;
```

3. Función: `fn_calcular_margen_ganancia(NUMERIC, NUMERIC)` FUNCIÓN MATEMÁTICA
Retorno: `NUMERIC` | **Parámetros:** `p_precio NUMERIC, p_costo NUMERIC`

Propósito: Evalúa la tasa porcentual de ganancia bruta sobre el precio de venta evitando divisiones entre cero.

```
CREATE OR REPLACE FUNCTION fn_calcular_margen_ganancia(p_precio NUMERIC, p_costo NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    IF p_precio IS NULL OR p_precio ⩽ 0 THEN RETURN 0.00; END IF;
    RETURN ROUND(((p_precio - COALESCE(p_costo, 0)) / p_precio) * 100, 2);
END;
$$ LANGUAGE plpgsql;
```

4. Función: `fn_total_ventas_vendedor(INT)` FUNCIÓN DE AGREGACIÓN
Retorno: `NUMERIC` | **Parámetros:** `p_id_vendedor INT`

Propósito: Calcula el acumulado histórico de ventas liquidadas para liquidaciones o comisiones de un vendedor específico.

```
CREATE OR REPLACE FUNCTION fn_total_ventas_vendedor(p_id_vendedor INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COALESCE(SUM(dv.subtotal), 0) INTO v_total
    FROM detalle_venta dv
    JOIN productos p ON dv.id_producto = p.id_producto
    WHERE p.id_vendedor = p_id_vendedor;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

5. Función: `fn_consultar_pedidos_cliente(INT)` FUNCIÓN DE CONJUNTO / TABLA
Retorno: `TABLE(id_pedido INT, fecha_pedido DATE, total NUMERIC, estado VARCHAR, articulos_distintos BIGINT)`

Propósito: Retorna el historial estructurado de compras de un usuario para su perfil o la aplicación móvil.

```
CREATE OR REPLACE FUNCTION fn_consultar_pedidos_cliente(p_id_comprador INT)
RETURNS TABLE (
    id_pedido INT,
    fecha_pedido DATE,
    total NUMERIC,
    estado VARCHAR,
    articulos_distintos BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        pv.id_pedido, pv.fecha_pedido, pv.total, pv.estado,
        COUNT(dv.id_detalle) AS articulos_distintos
    FROM pedidos_ventas pv
    LEFT JOIN detalle_venta dv ON pv.id_pedido = dv.id_pedido
    WHERE pv.id_comprador = p_id_comprador
    GROUP BY pv.id_pedido, pv.fecha_pedido, pv.total, pv.estado
    ORDER BY pv.fecha_pedido DESC;
END;
$$ LANGUAGE plpgsql;
```

6 Matriz de Trazabilidad y Modelo Entidad-Relación

La siguiente matriz de impacto ilustra la relación bidireccional entre las tablas principales del modelo relacional de LocalMarket y los objetos avanzados implementados:

TABLA DEL SISTEMA	VISTAS VINCULADAS	PROCEDIMIENTOS	TRIGGERS & FUNCIONES

PRODUCTOS	`vw_catalogo_productos_detallado`
	`vw_alerta_stock_critico`
		`sp_procesar_venta`
		`sp_reabastecer_stock`
			`trg_evitar_stock_negativo`
			`trg_validar_precios_coherentes`
			`trg_auditar_agotamiento_stock`

PEDIDOS_VENTAS	`vw_resumen_pedidos_ventas`	`sp_crear_pedido` `sp_actualizar_estado_pedido`	Actualizada reactivamente por `trg_actualizar_total_pedido`
DETALLE_VENTA	`vw_resumen_pedidos_ventas` `vw_balance_financiero_tienda`	`sp_procesar_venta`	`trg_actualizar_total_pedido` `fn_calcular_subtotal_producto`
USUARIOS	`vw_metricas_vendedores`	`sp_crear_pedido`	`trg_normalizar_correo_usuario` `fn_total_ventas_vendedor`
VERIFICACIONES_OTP	Auditoría interna de seguridad	`sp_depurar_intentos_otp`	Control de tokens de doble factor

Diagrama Entidad-Relación de Soporte

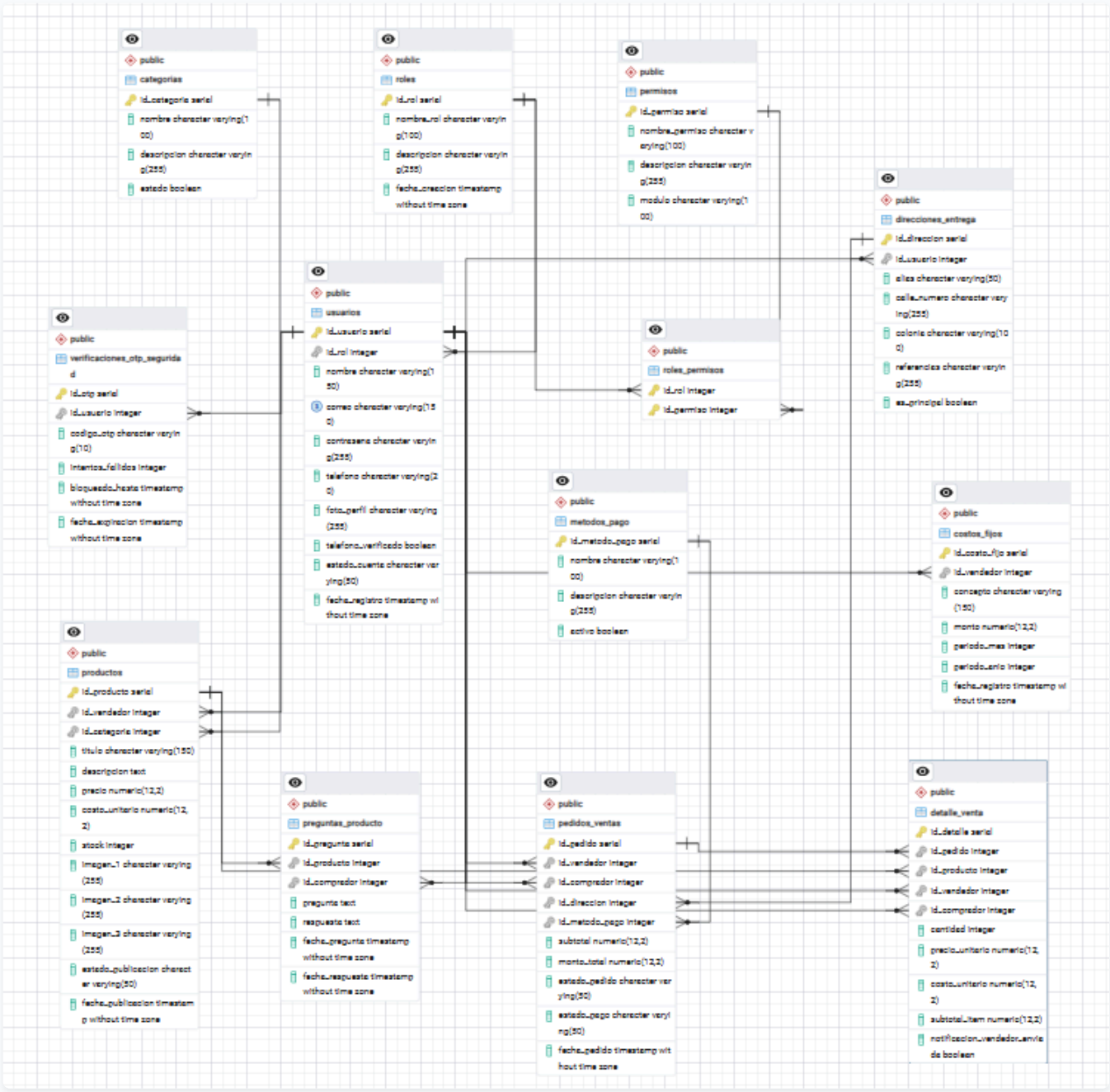


Figura 3. Esquema Entidad-Relación (13 tablas) base para la implementación de los objetos en PostgreSQL.

7 Conclusiones y Validación en Entorno de Producción

La incorporación de estas **5 vistas, 5 procedimientos, 5 triggers y 5 funciones** eleva la base de datos de **LocalMarket** a un estándar profesional de nivel empresarial:

- **Desacoplamiento y Rendimiento:** La lógica de descuento de inventario, cálculo de subtotales y validación de correos se procesa en microsegundos dentro del propio motor C/C++ de PostgreSQL, aligerando la carga del servidor Node.js.
- **Blindaje Transaccional:** Los triggers impiden estados inconsistentes como stocks negativos o márgenes comerciales negativos sin importar si la inserción proviene de la API REST, de un script externo o de una consulta manual.
- **Código Fuente Disponible:** El script SQL unificado se encuentra archivado en el proyecto como `scripts/objetos_bd.sql`, listo para despliegues automatizados mediante CI/CD.